

Practical 12 - Development and evaluation of risk prediction models

Survival Analysis

London School of Hygiene & Tropical Medicine

SOLUTIONS

Datasets required: rotterdam_p10.csv

R packages required: survival, pec, riskRegression, rms

```
library(survival)
library(pec)
library(riskRegression)
library(rms)
```

Introduction

In this practical we will perform analyses similar to those shown in the lecture, using the Rotterdam breast cancer data. The aim is to develop a prediction model for 5-year risk of the composite outcome of disease recurrence or death from any cause, and to evaluate the predictive performance of the model.

The data used is the same as in Practicals 10 and 11. The variables we will use are summarised in the table below.

Variable name	Description
pid	Patient identifier
year_cat	Year of surgery. 0: before 1990, 1: 1990 or later
age	Age at surgery
meno	Menopausal status. 0=premenopausal, 1=postmenopausal

Variable name	Description
<code>size</code>	Tumour size: categorical with levels ≤ 20 mm, 20 – 50mm, > 50 mm
<code>grade</code>	Differentiation grade: categorical with levels 2 and 3
<code>enodes</code>	Number of positive lymph nodes, transformed as <code>enodes=exp(-0.12*nodes)</code>
<code>pgr</code>	Progesterone receptors (fmol/l)
<code>er</code>	Estrogen receptors (fmol/l)
<code>chemo</code>	Chemotherapy: 0=no, 1=yes
<code>hormon</code>	Hormonal treatment: 0=no, 1=yes
<code>time</code>	Years to the first of recurrence or death without recurrence
<code>outcome</code>	Indicator of composite outcome: 0=censored, 1=event
<code>outcome_cr</code>	Competing events outcome: 0=censored, 1=recurrence, 2=death without recurrence

Aims

The aims of this session are:

- Fit prediction models and evaluate their performance.
- Evaluate prediction model performance using measures of calibration, discrimination, and overall performance.
- Use random split-sampling and k-fold cross-validation for evaluation of prediction model performance.

Question 1

- Load the data.
- It turns out to be sensible to change the scale of the variables `pgr` and `er`, so we will divide each of these by 100 for this analysis.
- Create a training set on which the model will be fitted, and a test set on which the predictive performance of the model will be evaluated. The training set is a random sample of 70% of the cohort, and the remaining 30% for the test set.

```
#load the data
roott<-read.table("datasets/rotterdam_p10.csv",sep="," ,header=T)

#divide pgr and er by 100
roott$pgr100<-roott$pgr/100
roott$er100<-roott$er/100
```

```
#Randomly split the data into training and test sets
#Use the same random seed so that we all get the same answer
set.seed(1972)

samp<-sample(rottpid, 2087, replace=F) #0.7*2982=2087.4

train<-rottpid[samp,]
test<-rottpid[!samp,]
```

Question 2

- Using the training set, fit a Cox regression for the composite outcome using the following predictors: `hormon, chemo, age, meno, size, grade, enodes, pgr100, er100, year_cat`. Use the `x=TRUE` option in `coxph`, as this may be helpful for some of the functions we will use later on.
- Use the model to obtain estimates of 5-year risk for individuals in the test set (e.g. using `predictSurvProb`).
- What is the range of the predicted risks in the test set?

SOLUTIONS

```
#fit a prediction model including all variables, using training data
cox.pred<-coxph(Surv(time,outcome)~hormon+chemo+age+meno+size+
               as.factor(grade)+enodes+pgr100+er100+year_cat,
               data=train,x=TRUE)

#risk predictions in the test data
#Using predictSurvProb from pec package
risk.test<-1-predictSurvProb(cox.pred,newdata=test,times=5)

summary(risk.test)
```

```
V1
Min.   :0.1051
1st Qu.:0.3019
Median :0.3886
Mean    :0.4402
3rd Qu.:0.5438
```

Max. :0.9872

The estimates of 5-year risk for individuals in the test set range from 0.105 to 0.987.

Question 3

In this question we will look at two measures of calibration: mean calibration and the calibration slope. These are obtained for the test data. The code is given, but you can probably do all of this without looking at it.

Mean calibration

- Calculate the mean estimated 5-year risk in the test data, and compare this with the Kaplan-Meier estimate of risk up to 5 years.
- What is the ratio of the observed to the expected risk at 5 years?
- The solution shows how to obtain a 95% confidence interval for this ratio.

Calibration slope

- Calculate the linear predictor from the risk model for each individual.
- Perform a Cox regression of the outcome on this linear predictor.
- What is the resulting coefficient? This is the calibration slope.

```
#---  
#MEAN CALIBRATION  
  
#mean 5-year risk  
exprisk.test<-mean(risk.test)  
exprisk.test  
  
#Kaplan-Meier estimate of 5-year risk  
km.test<-survfit(Surv(time,outcome)~1,data=test)  
obsrisk.test<-1-summary(km.test,times = 5)$surv  
obsrisk.test  
  
#ratio of observed/expected risk  
OEratio.test<-obsrisk.test/exprisk.test  
OEratio.test
```

```
#---
#CALIBRATION SLOPE

#linear predictor
lp.test<-predict(cox.pred,newdata=test,type="lp")

#Cox model including lp.test
calib.test<- coxph(Surv(time,outcome) ~ lp.test, data=test)
summary(calib.test)
confint(calib.test)
```

SOLUTIONS

```
#---
#MEAN CALIBRATION

#mean 5-year risk
exprisk.test<-mean(risk.test)
exprisk.test
```

```
[1] 0.4401978
```

```
#Kaplan-Meier estimate of 5-year risk
km.test<-survfit(Surv(time,outcome)~1,data=test)
obsrisk.test<-1-summary(km.test,times = 5)$surv
obsrisk.test
```

```
[1] 0.4204084
```

```
#ratio of observed/expected risk
OEratio.test<-obsrisk.test/exprisk.test
OEratio.test
```

```
[1] 0.9550443
```

```
#95% CI for OEratio.test
OEratio.test.lowerci<-OEratio.test*
  exp(-qnorm(1 - 0.05/2)*
    sqrt(1 / summary(km.test,times = 5)$n.event))
OEratio.test.upperci<-OEratio.test*
  exp(qnorm(1 - 0.05/2)*
    sqrt(1 / summary(km.test,times = 5)$n.event))
OEratio.test.lowerci
```

```
[1] 0.8628792
```

```
OEratio.test.upperci
```

```
[1] 1.057054
```

The mean 5-year risk estimated from the model is 0.440. The Kaplan-Meier estimate of 5-year risk is 0.420. The ratio of these is 0.955, with 95% CI 0.863 to 1.057. The ratio of observed to expected 5-year risk is close to 1, and the 95% CI includes 1. These results tell us that the mean calibration is good.

```
#---
#CALIBRATION SLOPE

#linear predictor
lp.test<-predict(cox.pred,newdata=test,type="lp")

#Cox model including lp.test
calib.test<- coxph(Surv(time,outcome) ~ lp.test, data=test)
summary(calib.test)
```

Call:

```
coxph(formula = Surv(time, outcome) ~ lp.test, data = test)
```

```
n= 895, number of events= 373
```

```
      coef exp(coef) se(coef)      z Pr(>|z|)
lp.test 0.98869   2.68771  0.07654 12.92  <2e-16 ***
```

```
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```

      exp(coef) exp(-coef) lower .95 upper .95
lp.test      2.688      0.3721      2.313      3.123

```

```

Concordance= 0.682 (se = 0.014 )
Likelihood ratio test= 150.9 on 1 df,  p=<2e-16
Wald test              = 166.8 on 1 df,  p=<2e-16
Score (logrank) test = 176 on 1 df,  p=<2e-16

```

```
confint(calib.test)
```

```

      2.5 %    97.5 %
lp.test 0.8386639 1.138715

```

The coefficient for the linear predictor in a Cox regression on the linear predictor values LP is 0.989, with 95% CI 0.839 to 1.139. The estimated calibration slope is close to 1, and the 95% CI includes 1. This is another indicator that the calibration is good.

Question 4

Follow the steps below to create a calibration plot:

- Divide the predicted risks in the test data into 10 equal-sized groups (by placing cut-points at the deciles).
- Calculate the mean of the predicted risks for the people in each of the 10 groups.
- Obtain the Kaplan-Meier estimate of 5-year risk in each group (and also the 95% confidence bounds if you like).
- Plot the Kaplan-Meier estimates against the mean predicted risks.

Discuss what the plot shows.

```

#divide predicted risks into 10 equal sized groups
risk.groups<-cut(risk.test,
                 breaks=quantile(risk.test,probs=seq(0,1,0.1)),
                 include.lowest=T,labels = F)

#mean predicted risk in each group
riskpred.group.test<-sapply(1:10,
                             FUN=function(x){
                               mean(risk.test[risk.groups==x])
                             })

```

```

#observed risk in each group, plus confidence intervals
obsrisk.group.test<-rep(NA,10)
obsrisk.group.test.lowerci<-rep(NA,10)
obsrisk.group.test.upperci<-rep(NA,10)
for(i in 1:10){
  km.test.temp<-survfit(Surv(time,outcome)~1,data=test[risk.groups==i,])
  obsrisk.group.test[i]<-1-summary(km.test.temp,times = 5)$surv
  obsrisk.group.test.lowerci[i]<-1-summary(km.test.temp,times = 5)$lower
  obsrisk.group.test.upperci[i]<-1-summary(km.test.temp,times = 5)$upper
}

#plot observed versus predicted risk
plot(riskpred.group.test,obsrisk.group.test,type="b",
     xlim=c(0,1),ylim=c(0,1),
     xlab="Predicted",ylab="Observed",lwd=2)
for(i in 1:10){
  segments(riskpred.group.test[i],obsrisk.group.test.lowerci[i],
           riskpred.group.test[i],obsrisk.group.test.upperci[i])
}
abline(0,1,col="red", lwd = 2)

```

SOLUTIONS

```

#divide predicted risks into 10 equal sized groups
risk.groups<-cut(risk.test,
                 breaks=quantile(risk.test,probs=seq(0,1,0.1)),
                 include.lowest=T,labels = F)

#mean predicted risk in each group
riskpred.group.test<-sapply(1:10,
                             FUN=function(x){
                               mean(risk.test[risk.groups==x])
                             })

#observed risk in each group, plus confidence intervals
obsrisk.group.test<-rep(NA,10)
obsrisk.group.test.lowerci<-rep(NA,10)
obsrisk.group.test.upperci<-rep(NA,10)
for(i in 1:10){
  km.test.temp<-survfit(Surv(time,outcome)~1,data=test[risk.groups==i,])

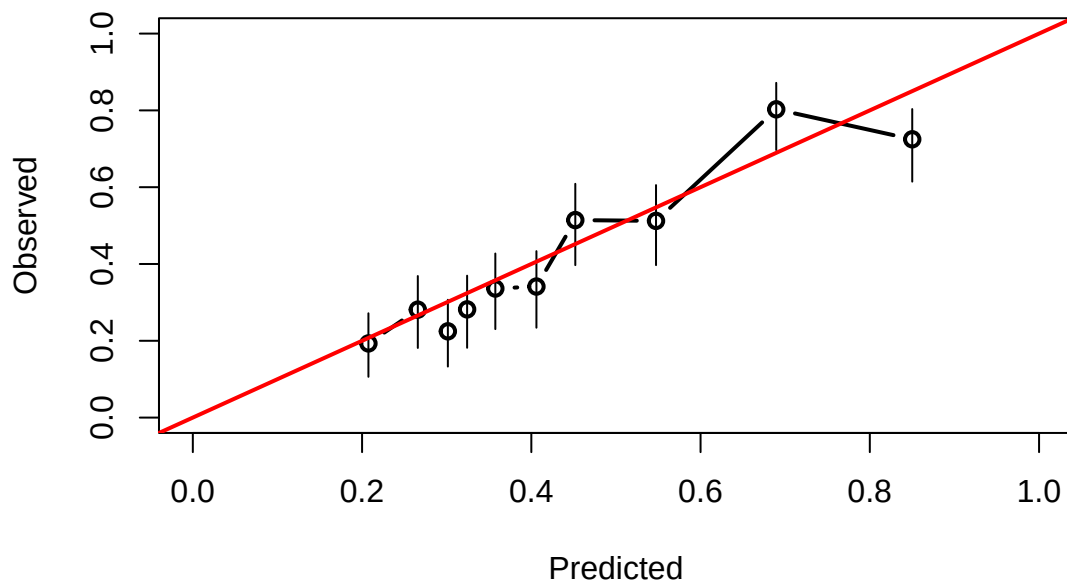
```

```

obsrisk.group.test[i]<-1-summary(km.test.temp,times = 5)$surv
obsrisk.group.test.lowerci[i]<-1-summary(km.test.temp,times = 5)$lower
obsrisk.group.test.upperci[i]<-1-summary(km.test.temp,times = 5)$upper
}

#plot observed versus predicted risk
plot(riskpred.group.test,obsrisk.group.test,type="b",
     xlim=c(0,1),ylim=c(0,1),
     xlab="Predicted",ylab="Observed",lwd=2)
for(i in 1:10){
  segments(riskpred.group.test[i],obsrisk.group.test.lowerci[i],
          riskpred.group.test[i],obsrisk.group.test.upperci[i])
}
abline(0,1,col="red", lwd = 2)

```



The observed risks within the 10 groups lie approximately on the 45-degree line when plotted against the mean predicted risks within the 10 groups. This indicates good calibration. The plot shows that people with low predicted risk were also observed to have low risk (looking at the Kaplan-Meier estimates of risk in the low predicted risk groups), and people with high predicted risk were also observed to have high risk (looking at the Kaplan-Meier estimates of risk in the high predicted risk groups).

Question 5

In this question we create a smooth version of the calibration plot in Question 4. Use the code below to implement the following steps (all in the test data):

- Calculate $Z_i = \log\{-\log(1 - \widehat{R}_i(5))\}$ for each person in the test data, where $\widehat{R}_i(5)$ is the estimated 5-year risk.
- Perform a Cox regression including Z_i as the only covariate, using a spline (this is done using a restricted cubic spline in the code). This is called the ‘refitted model’.
- Obtain estimates of risk up to 5 years for each person based on the refitted model including Z_i .
- Plot the estimated risks from the refitted model against the estimated risks from the main prediction model.

```
#generate log{-log(1-risk)}
test$loglogpred.test <- log(-log(1 - risk.test))

#Fit Cox regression of this using a spline:
#we do this here using the rms package (there are several other options)
calibcox.test<-cph(Surv(time,outcome) ~ rcs(loglogpred.test, 3),
                  data = test,x=T,surv=T,y=T)

#survival predictions from this model at time 5
calib.test<-survest(calibcox.test,newdata=test,times=5)

#make plot
calibplot.test<-data.frame(pred=risk.test,obs=1-calib.test$surv,
                          lower=1-calib.test$lower,upper=1-calib.test$upper)

calibplot.test<-calibplot.test[order(calibplot.test$pred),]

plot(
  calibplot.test$pred,
  calibplot.test$obs,
  type="l",
  lty = 1,
  xlim = c(0, 1),
  ylim = c(0, 1),
  lwd = 2,
  xlab = "Predicted risk from developed model",
  ylab = "Predicted risk from refitted model"
)
lines(calibplot.test$pred,
```

```

        calibplot.test$lower,
        type="l",
        lty = 2,
        lwd = 2)
lines(calibplot.test$pred,
      calibplot.test$upper,
      type="l",
      lty = 2,
      lwd = 2)
abline(0, 1, lwd = 2, lty = 1, col = 2)

```

SOLUTIONS

```

#generate log{-log(1-risk)}
test$loglogpred.test <- log(-log(1 - risk.test))

#Fit Cox regression of this using a spline:
#we do this here using the rms package (there are several other options)
calibcox.test<-cph(Surv(time,outcome) ~ rcs(loglogpred.test, 3),
                  data = test,x=T,surv=T,y=T)

#survival predictions from this model at time 5
calib.test<-survest(calibcox.test,newdata=test,times=5)

#make plot
calibplot.test<-data.frame(pred=risk.test,obs=1-calib.test$surv,
                          lower=1-calib.test$lower,upper=1-calib.test$upper)

calibplot.test<-calibplot.test[order(calibplot.test$pred),]

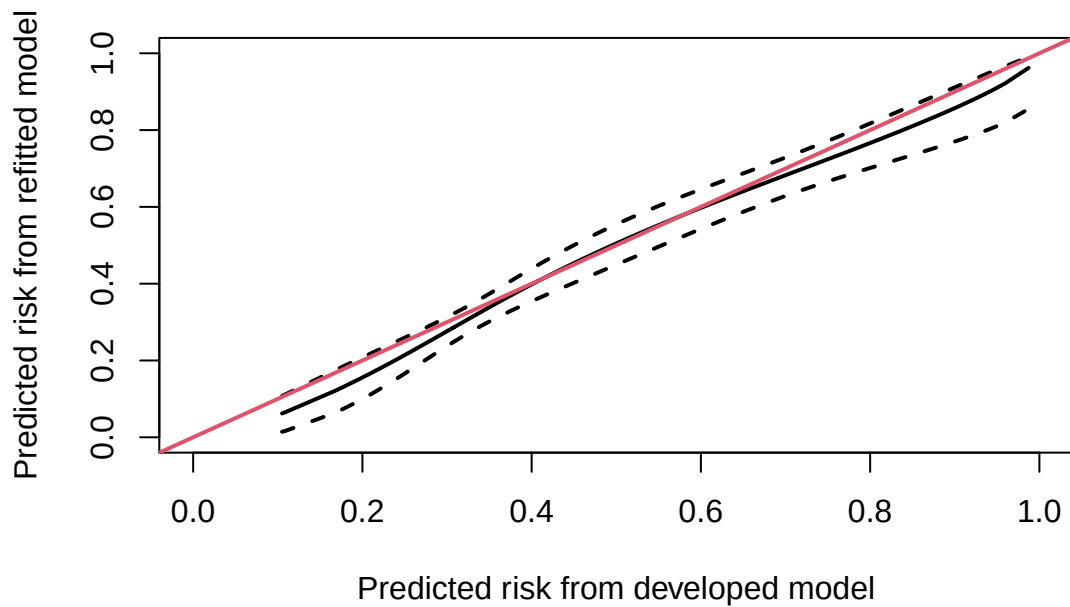
plot(
  calibplot.test$pred,
  calibplot.test$obs,
  type="l",
  lty = 1,
  xlim = c(0, 1),
  ylim = c(0, 1),
  lwd = 2,
  xlab = "Predicted risk from developed model",

```

```

    ylab = "Predicted risk from refitted model"
  )
  lines(calibplot.test$pred,
        calibplot.test$lower,
        type="l",
        lty = 2,
        lwd = 2)
  lines(calibplot.test$pred,
        calibplot.test$upper,
        type="l",
        lty = 2,
        lwd = 2)
  abline(0, 1, lwd = 2, lty = 1, col = 2)

```



This plot is a smoother version of that seen in the previous question, and it shows that the calibration is good because the smoothed black curve lies close to the 45-degree line.

Question 6

In this question we look at measures of discrimination.

- Use the code below to obtain estimates of Harrell's c-index and Uno's C-index. See the solution for how to obtain 95% CIs.
- Interpret the estimates.

```
#Harrell's C
Cindex.test<- concordance(Surv(time,outcome) ~ risk.test,
                          data=test,reverse = TRUE)
Cindex.test$concordance

#Uno's C
UnoC.test<- concordance(Surv(time,outcome) ~ risk.test,
                        data=test,timewt = "n/G2",reverse = TRUE)
UnoC.test$concordance
```

SOLUTIONS

```
#---
#Harrell's C
Cindex.test<- concordance(Surv(time,outcome) ~ risk.test,
                          data=test,reverse = TRUE)

#95% CIs
Cindex.test.lowerCI<-Cindex.test$concordance-qnrm(1 - 0.05/2)*
  sqrt(Cindex.test$var)
Cindex.test.upperCI<-Cindex.test$concordance+qnrm(1 - 0.05/2)*
  sqrt(Cindex.test$var)

#identical
Cindex.test.lp<- concordance(Surv(time,outcome) ~ lp.test,
                             data=test,reverse = TRUE)

paste0(round(Cindex.test$concordance,3)," (",
        round(Cindex.test.lowerCI,3)," ",round(Cindex.test.upperCI,3),")")
```

```
[1] "0.682 (0.654, 0.709)"
```

```
#---
#Uno's C
UnoC.test<- concordance(Surv(time,outcome) ~ risk.test,
                        data=test,timewt = "n/G2",reverse = TRUE)

#95% CIs
UnoC.test.lowerCI<-UnoC.test$concordance-qnrm(1 - 0.05/2)*
```

```

sqrt(UnoC.test$var)
UnoC.test.upperCI<-UnoC.test$concordance+qnorm(1 - 0.05/2)*
sqrt(UnoC.test$var)

paste0(round(UnoC.test$concordance,3)," (",
        round(UnoC.test.lowerCI,3),", ",round(UnoC.test.upperCI,3),")")

```

```
[1] "0.681 (0.654, 0.709)"
```

The estimate of Harrell's c-index is 0.682 and the estimate of Uno's c-index is 0.681. Uno's c-index accounts for censoring using weights, whereas Harrell's c-index ignores censoring. In this example the numbers are very similar. The estimated c-index is fairly low, indicating that the model does not have very good discrimination, in other words the ability of the model to rank people correctly in terms of their risk is not that good.

We can also get the c-index using `cindex` from the `pec` package - see below.

```

cindex.test <- cindex(
  object = list("Cox" = cox.pred),
  formula = Surv(time, outcome) ~ 1,
  data = test,
  splitMethod = "none",
  # Use cens.model="none" for Harrell's C-index
  # Use cens.model="marginal" for Uno's C-index
  cens.model = "marginal"
)
cindex.test

```

The c-index for right censored event times

Prediction models:

Cox
Cox

Right-censored response of a survival model

No.Observations: 895

Pattern:

	Freq
event	373
right.censored	522

Censoring model for IPCW: marginal model (Kaplan-Meier for censoring distribution)

No data splitting: either apparent or independent test sample performance

Estimated C-index in % at time=5

	AppCindex	Pairs Concordant
Cox	68.1	531571 362085

Warning in summary.Cindex(x, print = TRUE, ...): The C-index is not proper for t-year predictions. Blanche et al. (2018), Biostatistics, 20(2): 347--357.

Consider using time-dependent AUC instead: riskRegression::Score

Question 7

Lastly we look at the Brier score, which is a measure of overall performance.

- Use the code below to obtain an estimate of the Brier score and the scaled Brier score.
- Interpret the estimates. What does the scaled Brier score compare?

```
#Brier score: using Score from riskRegression package
brier.test <- Score(list("cox" = cox.pred),
  formula = Surv(time, outcome)~1,
  data = test,
  conf.int = TRUE,
  times = 4.99,
  #The time should be just before the time horizon of interest
  #(unless there happens to be an event at that time)
  cens.model = "km",
  metrics = "brier",
  summary = "ipa")

brier.test$Brier$score
```

SOLUTIONS

```
#Brier score: using Score from riskRegression package
brier.test <- Score(list("cox" = cox.pred),
                     formula = Surv(time, outcome)~1,
                     data = test,
                     conf.int = TRUE,
                     times = 4.99,
                     #The time should be just before the time horizon of interest
                     #(unless there happens to be an event at that time)
                     cens.model = "km",
                     metrics = "brier",
                     summary = "ipa")

brier.test$Brier$score
```

	model	times	Brier	se	lower	upper	IPA
	<fctr>	<num>	<num>	<num>	<num>	<num>	<num>
1:	Null model	4.99	0.2436652	0.002644179	0.2384827	0.2488477	0.0000000
2:	cox	4.99	0.2077455	0.005961543	0.1960611	0.2194299	0.1474141

The Brier score based on the Cox model is 0.208 and the Brier score based on the null model (with no predictors) is 0.244. The scaled Brier score ('IPA') is 0.147, indicating that the Brier score is reduced by 14.7% for the prediction model compared with the null model.

Question 8

So far we fitted the prediction model in the training data and evaluated the model performance in the test data. A better approach is to use k-fold cross validation. As a reminder, this involves the following steps, for 10-fold cross-validation:

- Divide the full data (i.e. before it was split in to training and test sets) randomly into approximately 10 equal-sized groups ('folds').
- Exclude fold 1 and fit the model in folds 2-10. Then obtain risk predictions in fold 1.
- Exclude fold 2 and fit the model in the remaining 9 folds. Then obtain predictions in fold 2.
- ...and so on, excluding each fold one by one. This results in a cross-validated risk prediction for each individual.
- Measures of predictive performance are then obtained using the cross-validated risk predictions on the full data.

k-fold cross-validation can be done automatically using the `Score` function in the `riskRegression` package as follows. This gives several measures of predictive performance, and enables plots. It doesn't give estimates of the c-index, but these can be obtained using `cindex` in the `pec` package, which uses bootstrap cross-validation rather than k-fold cross-validation.

```
#10-fold cross-validation using Score from riskRegression
eval.cv <- Score(
  object = list("Cox" = cox.pred),
  #More than one model can be listed in object if you want to compare
  formula = Surv(time, outcome) ~ 1,
  data = rott,
  split.method = "cv10",
  B = 1,
  metrics=c("AUC","Brier"),
  summary="ipa",
  plots="calibration",
  times = 4.99
)

plotCalibration(eval.cv)

#bootstrap cross-validation for the C-index using pec
cindex.boot <- cindex(
  object = list("Cox" = cox.pred),
  formula = Surv(time, outcome) ~ 1,
  data = rott,
  splitMethod = "bootcv",
  cens.model = "marginal",
  confInt = T #Doesn't seem to work yet
)
```

SOLUTIONS

```
#10-fold cross-validation using Score from riskRegression
eval.cv<-Score(
  object = list("Cox" = cox.pred),
  #More than one model can be listed in object if you want to compare
  formula = Surv(time, outcome) ~ 1,
  data = rott,
```

```

split.method = "cv10",
B = 1,
metrics=c("AUC","Brier"),
summary="ipa",
plots="calibration",
times = 4.99,
progress.bar = F
)

```

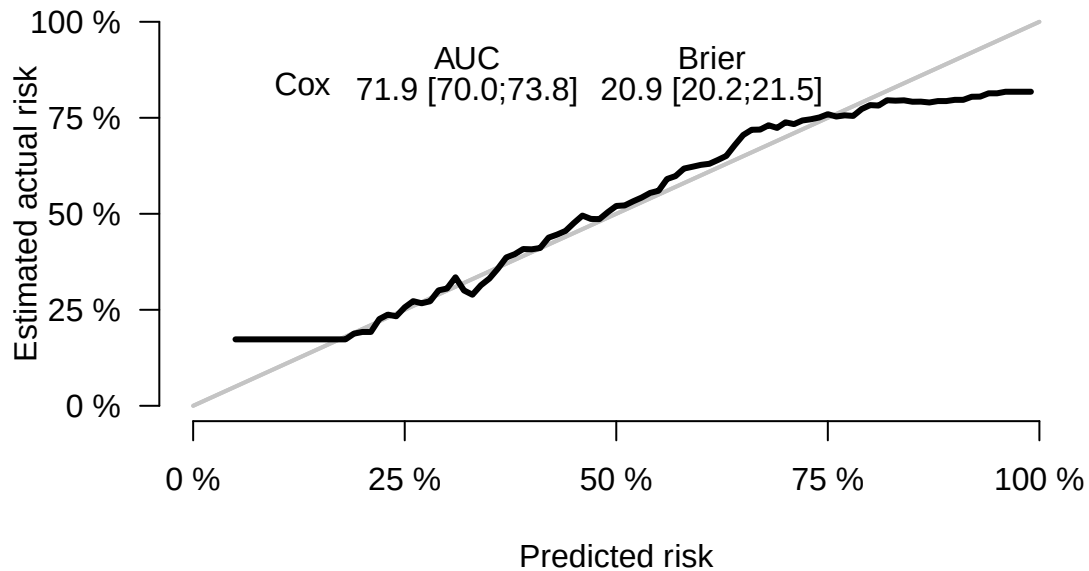
```

|
|                                     | 0%

|
|==                                | 10%
|
|====                             | 20%
|
|=====                          | 30%
|
|=====                          | 40%
|
|=====                          | 50%
|
|=====                          | 60%
|
|=====                          | 70%
|
|=====                          | 80%
|
|=====                          | 90%
|
|=====                          | 100%

```

```
plotCalibration(eval.cv)
```



```
> eval.cv
```

Metric AUC:

Results by model:

	model	times	AUC	lower	upper
	<fctr>	<num>	<char>	<char>	<char>
1:	Cox	5	72.1	70.2	74.0

NOTE: Values are multiplied by 100 and given in %.

NOTE: The higher AUC the better.

Metric Brier:

Results by model:

	model	times	Brier	lower	upper	IPA
	<fctr>	<num>	<char>	<char>	<char>	<char>
1:	Null model	4.99	24.6	24.4	24.8	0.0
2:	Cox	4.99	20.8	20.2	21.5	15.4

Results of model comparisons:

times	model	reference	delta.Brier	lower	upper	p
<num>	<fctr>	<fctr>	<char>	<char>	<char>	<num>

```
1:  4.99    Cox Null model      -3.8   -4.4   -3.2 1.012586e-32
```

NOTE: Values are multiplied by 100 and given in %.

NOTE: The lower Brier the better, the higher IPA the better.

Data for Calibration plot are stored in the object as `x[["Calibration"]]`.

10-fold cross-validation repeated 1 times.

```
cindex.boot <- cindex(  
  object = list("Cox" = cox.pred),  
  formula = Surv(time, outcome) ~ 1,  
  data = rott,  
  splitMethod = "bootcv",  
  cens.model = "marginal",  
  confInt = T, #Doesn't seem to work yet  
  verbose=F  
)  
  
cindex.boot
```

The c-index for right censored event times

Prediction models:

Cox

Cox

Right-censored response of a survival model

No.Observations: 2982

Pattern:

	Freq
event	1285
right.censored	1697

Censoring model for IPCW: marginal model (Kaplan-Meier for censoring distribution)

Method for estimating the prediction error:

Bootstrap cross-validation

Type: resampling
Bootstrap sample size: 2982
No. bootstrap samples: 100
Sample size: 2982

Estimated C-index in % at time=5

	AppCindex	BootCvCindex
Cox	68.8	68.4

AppCindex : Apparent (training data) performance
BootCvCindex : Bootstrap crossvalidated performance

Warning in summary.Cindex(x, print = TRUE, ...): The C-index is not proper for t-year predictions. Blanche et al. (2018), Biostatistics, 20(2): 347--357.

Consider using time-dependent AUC instead: riskRegression::Score

- The estimated Brier score is 0.208 (95% CI 0.202 to 0.215), and the scaled Brier score is 15.4%.
- The calibration plot is less smooth than that obtained earlier (I think due to how the smoothing has been done, as well as the k-fold cross validation), but tells a similar story.
- The AUC reported by **Score** is another measure of discrimination, which is similar to (but not identical to) the c-index.
- The cross-validated c-index estimate is 0.686. Notice that **cindex** also reports the ‘apparent performance’ c-index (0.688), which is the c-index based on assessing the prediction model in the same data in which the model was fitted. As we expect, the cross-validated c-index is lower than the apparent performance c-index.

Question 9

- Suppose you were happy to proceed with your model on the basis of the above evaluations of predictive performance. Obtain and write down a formula that could be used to obtain a prediction for a new individual.

SOLUTIONS

```
#fit a prediction model including all variables, using the full data
cox.pred.final<-coxph(Surv(time,outcome)~hormon+chemo+age+meno+size+
                      as.factor(grade)+enodes+pgr100+er100+year_cat,
                      data=rott)

#---
#Components needed for the prediction model formula

#baseline cumulative hazard at time 5
H0<-max(basehaz(cox.pred.final,centered=F)$hazard)
S0<-exp(-H0)
S0
```

```
[1] 0.08712617
```

```
#same thing, using survfit
# newdata<-data.frame(hormon=0,chemo=0,age=0,meno=0,size("<=20",grade=2,
#                      enodes=0,pgr100=0,er100=0,year_cat=0)
# S0<-min(survfit(cox.pred.final,newdata=newdata)$surv)

#coefficients (beta parameters) in the linear predictor
cox.pred.final$coefficients
```

hormon	chemo	age	meno
-0.315488714	-0.330066254	-0.004409893	0.047479237
size>50	size20-50	as.factor(grade)3	enodes
0.591552212	0.361347523	0.348596688	-1.916631018
pgr100	er100	year_cat	
-0.023678558	-0.022292460	-0.163273661	

At the end, the chosen model should be refitted using all the data, i.e. using the training and test data combined. The formula for obtaining 5-year risk predictions is:

$$\widehat{R}(5) = 1 - \widehat{S}_0(5)^{\exp(LP)}$$

where the baseline survival probability at time 5 years is $\widehat{S}_0(5) = 0.08712617$ and the linear predictor LP is (with coefficients presented to 3 decimal places): $LP = -0.315 \times \text{hormon} - 0.330 \times \text{chemo} - 0.004 \times \text{age} + 0.047 \times \text{meno} + 0.361 \times I(\text{size} = 20 - 50) + 0.592 \times I(\text{size} > 50) + 0.348 \times I(\text{grade} = 3) - 1.917 \times \text{enodes} - 0.024 \times \text{pgr100} - 0.022 \times \text{er100} - 0.163 \times I(\text{year} > 1990)$.